

# CHAPTER 3: RESEARCH METHODOLOGY AND PROPOSED SYSTEM

## 3.1 Introduction

This chapter describes the research methodology used in the design and development of the Cloud-Based Examination Script Management System for the Computing Sciences Department of Koforidua Technical University. It explains the approach followed during the development process and highlights the major activities carried out to achieve the objectives of the project. The chapter also discusses how the selected methodology supported the successful implementation of the proposed system.

The chapter further explains the software development methodology adopted for the project, including the reasons for selecting it over other development approaches. It outlines the different phases involved in the development process and describes the activities performed during each phase, such as requirement gathering, system design, implementation, testing, deployment, and maintenance. The methodology provided a structured and flexible approach that ensured continuous improvement throughout the project.

In addition, the chapter presents the proposed system and its overall architecture. It discusses the major components, system features, development tools, and technologies used in building the application. The proposed system was designed to support secure examination script upload, cloud storage integration, PDF compression, and efficient retrieval of academic documents through a centralized web platform.

Furthermore, the chapter includes various UML models used to represent the structure and behavior of the proposed system. These models include the context diagram, sequence diagram, class diagram, and activity diagram, which illustrate the interactions between users and the system. The chapter also examines the strengths and possible limitations of the proposed solution to provide a clear understanding of its capabilities and operational challenges.

## 3.2 Software Development Methodology

### 3.2.1 Introduction to Rapid Application Development (RAD)

The **Rapid Application Development (RAD)** methodology was adopted for the development of this project. RAD is an iterative software development methodology that emphasizes rapid prototyping, continuous user involvement, and quick delivery of functional software components. Unlike traditional development models that require extensive planning before implementation,

RAD focuses on developing working prototypes that can be evaluated and refined through user feedback.

The methodology encourages frequent interaction between developers and users throughout the development process, allowing requirements to be refined continuously. This approach significantly reduces development time while ensuring that the final system meets user expectations. RAD is particularly suitable for web-based applications where requirements can be validated through prototype demonstrations and incremental improvements.

### **3.2.2 Justification for Choosing Rapid Application Development (RAD)**

The Rapid Application Development (RAD) methodology was selected because it supports the rapid development and continuous refinement of software through prototyping and user feedback. Since the Cloud-Based Examination Script Management System required multiple interactive modules such as authentication, script upload, cloud storage integration, and PDF processing, RAD provided an efficient framework for building and improving these features within a short development period.

The following reasons justify the use of RAD for this project:

#### **1. Rapid Development**

RAD enables the development of functional prototypes within a short period, allowing early demonstration of system features.

#### **2. User Participation**

Lecturers, departmental administrators, and prospective users provided continuous feedback throughout development, ensuring that the system satisfied academic requirements.

#### **3. Prototyping**

Functional prototypes were developed and refined repeatedly until the desired functionality and usability were achieved.

#### **4. Flexibility**

New requirements and feature improvements were easily incorporated without restarting the development process.

#### **5. Reduced Development Time**

The use of reusable components and incremental development accelerated the completion of the project.

#### **6. Improved System Quality**

Continuous testing and user evaluation during prototype development helped identify and correct errors early, resulting in a more reliable system.

### **3.2.3 Rapid Application Development (RAD) Process**

The RAD methodology used in this project consisted of the following phases:

1. Requirements Planning
2. User Design
3. Construction
4. Cutover

Each phase involved continuous interaction between developers and users to ensure that the developed system met the project's objectives and functional requirements.

## **3.3 Activities Performed in Each RAD Phase**

### **3.3.1 Requirements Planning**

This phase focused on identifying the objectives, system requirements, and scope of the proposed Cloud-Based Examination Script Management System. Information was gathered through literature review, observation of existing document management systems, and analysis of examination script management practices within educational institutions.

The following activities were carried out:

1. Identification of system users and stakeholders.
2. Definition of project objectives and scope.
3. Collection of functional requirements such as:
  - i. User registration and login.
  - ii. Examination script upload.
  - iii. PDF compression and optimization.
  - iv. Cloud storage integration.
  - v. Search and retrieval of scripts.
  - vi. Automated notifications.
  - vii. Time-based access control.

4. Collection of non-functional requirements such as:
  - i. Security.
  - ii. Reliability.
  - iii. Performance.
  - iv. Scalability.
  - v. Usability.
5. Planning of project timeline and sprint activities.

### **3.3.2 User Design**

During this phase, prototypes of the proposed system were designed and continuously improved based on user feedback. The aim was to create a system that met both functional and usability requirements before full implementation.

The following activities were carried out:

1. Designing the database schema using MySQL.
2. Designing the web application user interface.
3. Creating API structures for communication between frontend and backend.
4. Designing UML diagrams including context, sequence, class, and activity diagrams.
5. Designing authentication and authorization workflows.
6. Designing PDF compression and cloud upload processes.
7. Designing notification and access control mechanisms.

### **3.3.3 Construction**

This phase involved the actual development of the system using the approved prototypes from the User Design phase. Coding, testing, and refinement were carried out simultaneously to ensure rapid progress.

The following technologies were used:

1. React for frontend development.
2. Next.js and Express.js for backend API development.

3. MySQL for database management.
4. Cloud storage integration services.
5. Postman for API testing.
6. WebStorm IDE for development.

The following modules were implemented:

1. User Authentication Module.
2. Examination Script Upload Module.
3. PDF Compression Module.
4. Cloud Storage Module.
5. Script Retrieval and Search Module.
6. Notification and Alert Module.
7. Access Control and Authorization Module.
8. Administrative Management Module.

### **3.3.4 Cutover**

1. The Cutover phase involved final system testing, deployment, and preparation of the system for use. All components were integrated and evaluated to ensure that the application satisfied the project requirements.
2. The following testing methods were used:
  - 3. Unit Testing**
  4. Each module was tested independently to verify its functionality.
  - 5. Integration Testing**
  6. The interaction between the frontend, backend APIs, cloud storage services, and database was tested.
  - 7. System Testing**
  8. The complete application was tested to ensure that all modules functioned together correctly.
  - 9. User Acceptance Testing**

10. Lecturers and departmental administrators evaluated the completed system to verify that it satisfied academic and operational requirements.
11. The following tools were used during testing:
12. Postman for API testing.
13. Browser developer tools.
14. MySQL Workbench for database verification.
15. Following successful testing, the following deployment activities were carried out:
16. Backend server setup.
17. Database deployment.
18. Cloud storage configuration.
19. Frontend deployment.
20. Final system configuration.
21. After deployment, maintenance activities focused on correcting minor issues, improving system performance, and identifying future enhancements.
22. Possible future improvements include:
23. Integration with university academic systems.
24. AI-based document classification and indexing.
25. Mobile application support.
26. Real-time collaboration and moderation features.
27. Expansion to other departments within the university.
- 28.

## **3.4 Proposed System**

### **3.4.1 Introduction of the Proposed System**

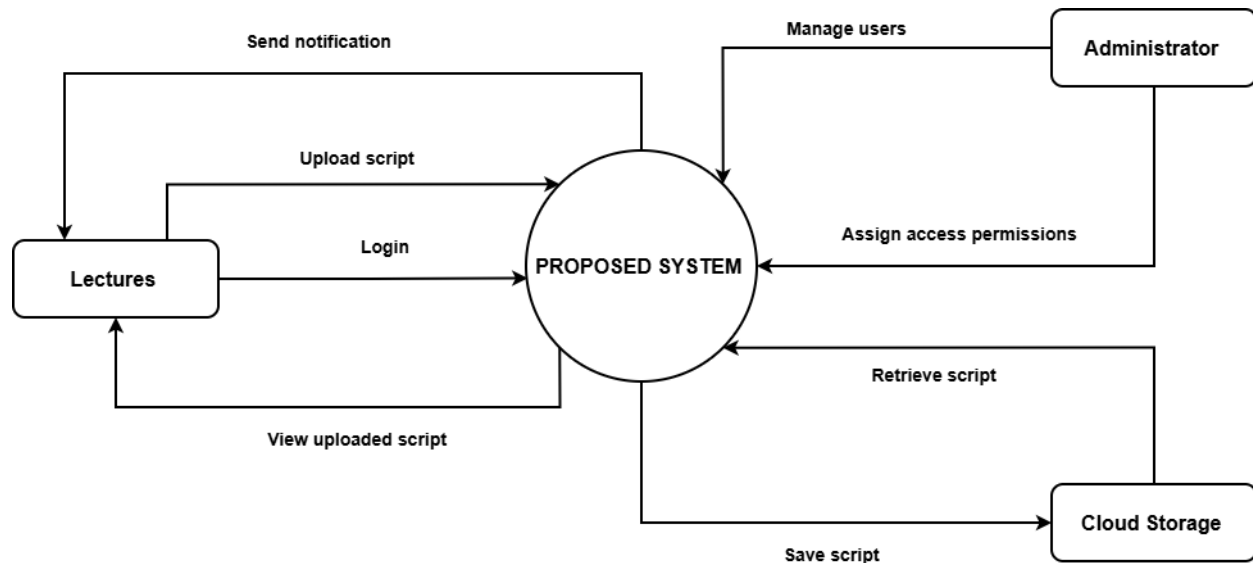
The proposed system is a Cloud-Based Examination Script Management System designed for the Computing Sciences Department of Koforidua Technical University. The system enables lecturers and administrators to upload, store, retrieve, and manage examination scripts electronically through a centralized web platform.

The system integrates PDF compression technology to reduce document sizes before cloud storage, thereby improving storage efficiency and upload performance. It also provides role-based and time-based access control to ensure that only authorized users can access examination scripts within approved periods.

The proposed system is intended to reduce the challenges associated with manual paper-based storage, improve document accessibility, enhance security, and support long-term preservation of academic records.

## 3.5 Models of the Proposed System

### 3.5.1 Context Model



*Figure 3.1*

#### Description

The context model for the system consists of several external entities that interact with the main platform to support its operations and services. Lecturers interact with the system by uploading examination scripts, viewing uploaded documents, and receiving notifications related to submission deadlines. Department administrators manage users, monitor uploaded scripts, assign access permissions, and oversee system activities.

Cloud storage services provide remote storage facilities for uploaded examination scripts and ensure secure file preservation and retrieval. Notification services enable the delivery of alerts and reminders to lecturers and administrators through email or in-system notifications. The database

server stores and manages all system data securely, including user information, metadata, access logs, and examination script records.

### 3.5.2 Interaction Model (Sequence Diagram)

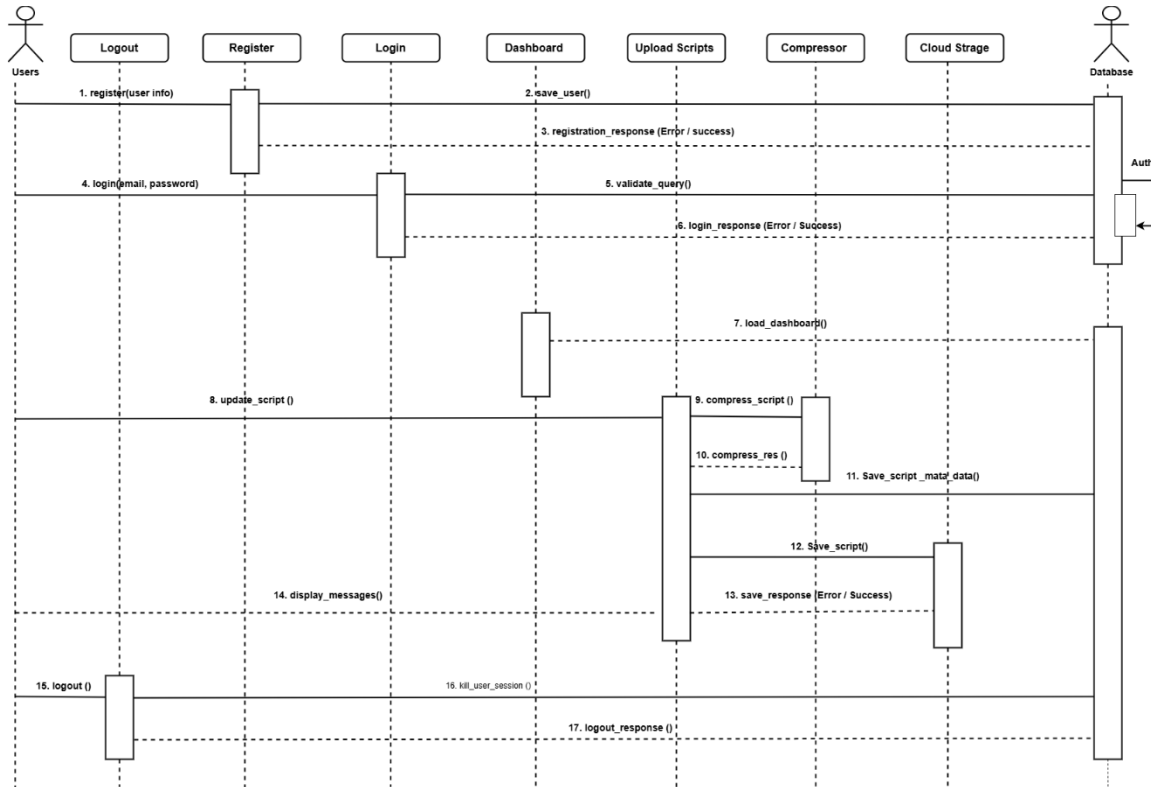


Figure 3.2

#### Description

The sequence flow of the system begins when the user opens the web application and logs into the system to access its services. After successful authentication, the user selects the option to upload an examination script and enters the required script details such as course code, semester, and academic year.

The user then uploads the PDF document, after which the system validates and compresses the file to optimize storage usage. Once processing is completed successfully, the file is uploaded to cloud storage while the related metadata is stored in the MySQL database.

After successful storage, the system sends notifications to administrators or relevant users. Finally, the user receives a confirmation message indicating that the upload process was completed successfully.



### 3.5.3 Structural Model (Class Diagram)

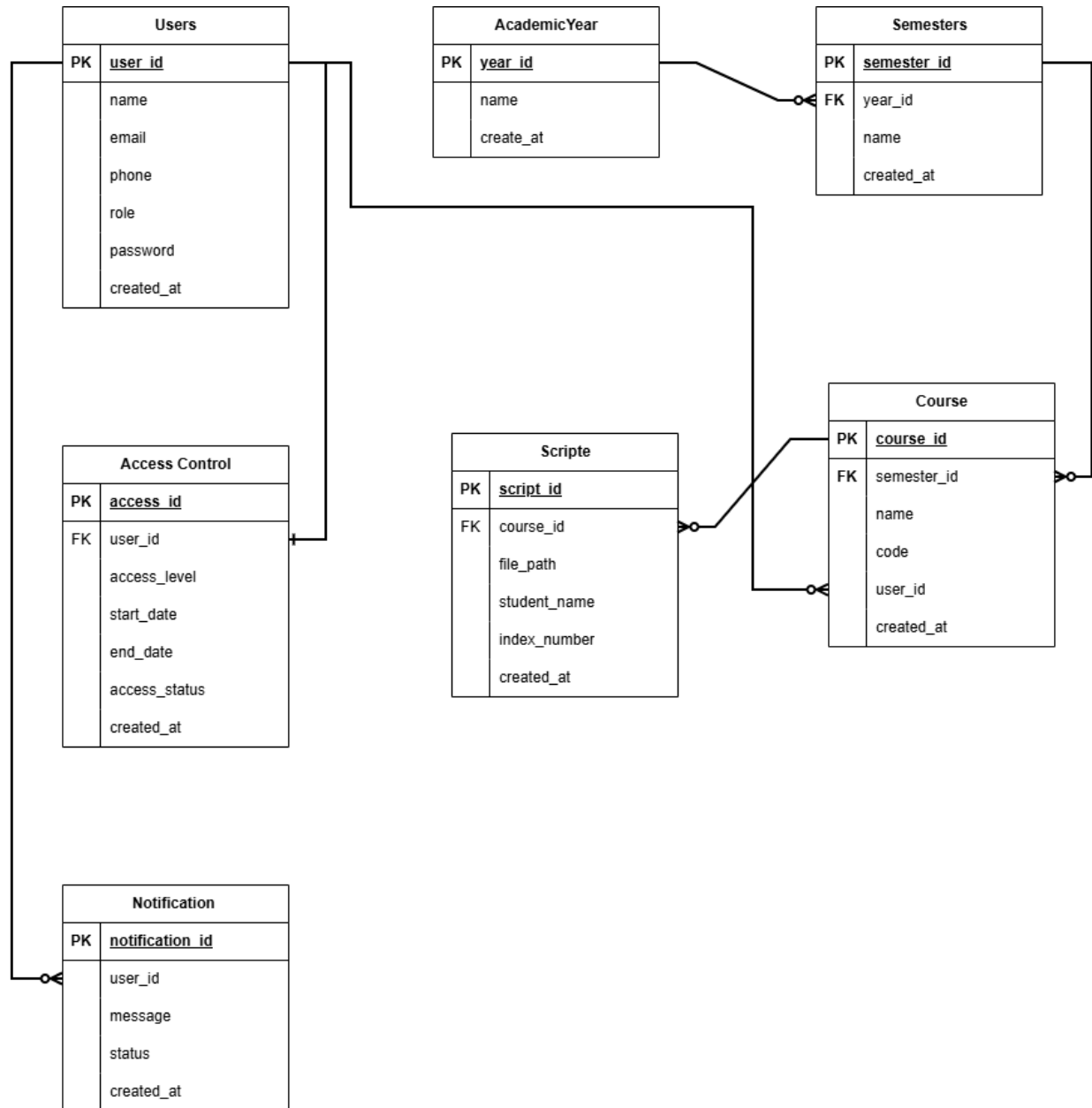


Figure 3.3

#### Description

The class structure of the system is composed of several main classes that work together to support examination script management functionalities. The User class stores user-related information such as UserID, Name, Email, Role, and Password for authentication and authorization purposes.

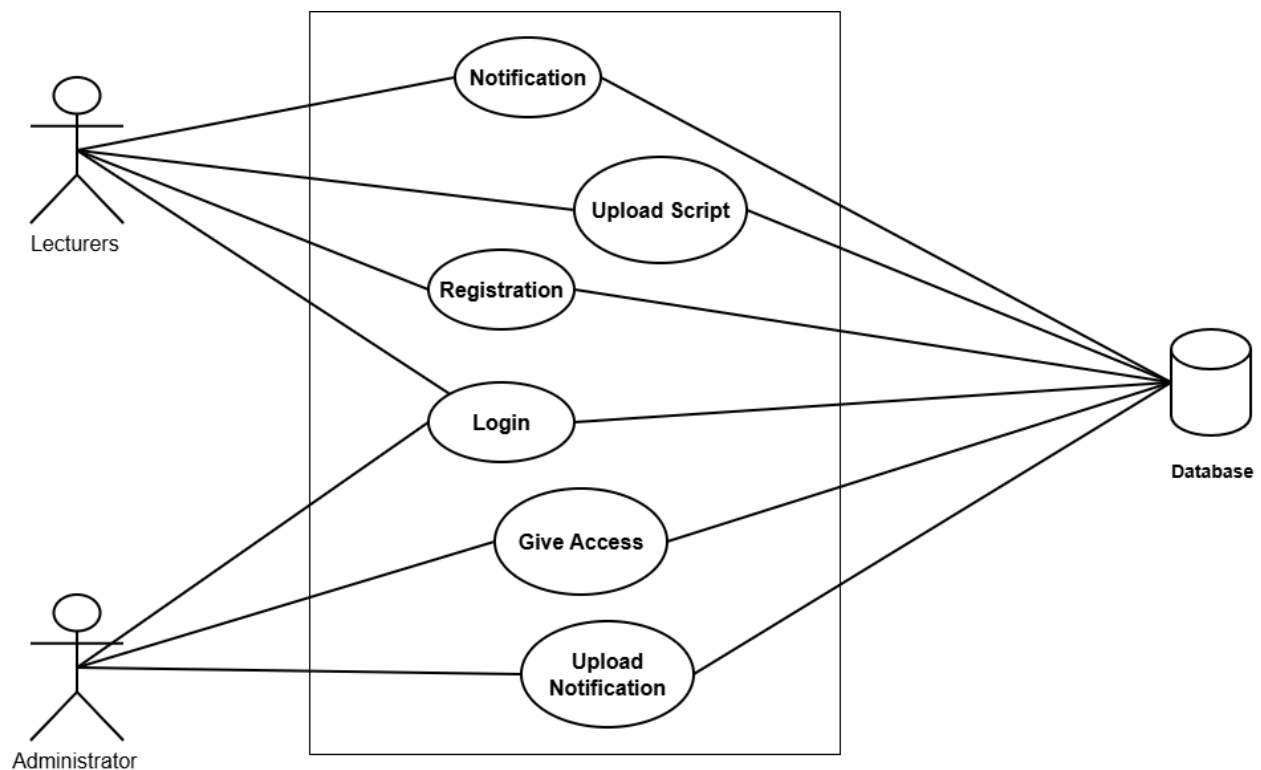
The ExaminationScript class manages uploaded examination documents and contains attributes including ScriptID, CourseCode, AcademicYear, Semester, FilePath, and UploadDate to support proper organization and retrieval of scripts.

The Notification class handles communication between the system and users by storing notification information such as NotificationID, Message, Status, and Timestamp. The AccessControl class manages role-based and time-based permissions using attributes such as AccessID, UserRole, AccessStartDate, and AccessEndDate.

Finally, the CloudStorage class manages cloud file information including StorageID, FileName, FileSize, and UploadStatus to support secure and efficient cloud storage operations.

### 3.5.4 Behavioral Model (Use Case Diagram)

The use case diagram illustrates the interaction between the lecturers, administrator, system database, and notification service within the Cloud-Based Examination Script Management System. The system is designed to support secure examination script upload, access management, notification handling, and document retrieval, with all activities connected to a centralized database.



*Figure 3.4*

## **Description**

The behavioral model of the Cloud-Based Examination Script Management System is represented using a use case approach to show the interactions between lecturers, administrators, the database, and notification services. The process begins when lecturers register and log into the web application to gain secure access to the platform. The administrator also logs into the system and grants access permissions to authorized lecturers.

After authentication, lecturers upload examination scripts by entering the required document details and selecting the PDF file for submission. The system validates and compresses the uploaded file before storing it securely in cloud storage. Metadata associated with the uploaded script is saved in the centralized database for easy management and retrieval.

Once the upload process is completed, notifications are automatically sent to the administrator to inform them of newly uploaded scripts, while lecturers receive confirmation feedback regarding successful uploads. Authorized users can later retrieve examination scripts using the search and retrieval functionality. Throughout the entire process, all operations and records are connected to and managed through the database to ensure security, accessibility, and proper document management.

## **3.6 Features of the Proposed System**

The proposed system includes the following features:

1. User registration and authentication.
2. Examination script upload functionality.
3. PDF compression and optimization.
4. Cloud-based document storage.
5. Secure script retrieval system.
6. Automated notifications and reminders.
7. Role-based access control.
8. Time-based authorization.
9. Search and indexing functionality.
10. Secure backend APIs.
11. User-friendly web interface.
12. Centralized database management.

13. File backup and recovery mechanisms.

## **3.7 Development Tools and Development Environment**

The following development tools and technologies were used in the implementation of the proposed system.

### **3.7.1 React**

React was used for developing the frontend user interface. It supports responsive and interactive web application development.

#### **Advantages**

1. Reusable components.
2. Fast rendering performance.
3. Responsive user interface.
4. Large community support.

### **3.7.2 Next.js and Express.js**

Next.js and Express.js were used for backend API development and server-side processing.

#### **Advantages**

1. Fast API development.
2. Scalable architecture.
3. Efficient routing and middleware support.
4. Easy integration with databases and cloud services.

### **3.7.3 MySQL Database**

MySQL was used as the database management system for storing user data, script metadata, notifications, and access control information.

#### **Advantages**

1. Reliable data storage.

2. Structured data management.
3. High performance.
4. Secure database operations.

### **3.7.4 Postman**

Postman was used for API testing and debugging.

#### **Uses**

1. Testing API endpoints.
2. Verifying request and response data.
3. Debugging backend services.
4. Monitoring API performance.

### **3.7.5 Cloud Storage Services**

Cloud storage services were used to store uploaded examination scripts securely.

#### **Uses**

1. Remote document storage.
2. File backup and recovery.
3. Secure file access.
4. Scalable storage management.

### **3.7.6 WebStorm IDE**

WebStorm IDE was used as the primary development environment for coding and managing the project.

#### **Advantages**

1. Intelligent code completion.
2. Debugging tools.
3. Integrated terminal.

4. Git integration.
5. Improved developer productivity.

### **3.8 Review of the Good Features of the Proposed System**

The proposed system offers several advantages that improve examination script management.

1. Centralized cloud storage improves document organization.
2. PDF compression reduces storage consumption.
3. Automated notifications improve submission compliance.
4. Secure authentication protects academic records.
5. Role-based access control improves confidentiality.
6. Fast retrieval improves efficiency in accessing scripts.
7. Backup and recovery mechanisms reduce data loss risks.
8. Agile development ensured continuous improvement and testing.
9. Reduced dependence on paper-based storage systems.

### **3.9 Review of the Bad Features of the Proposed System**

Despite its advantages, the proposed system has some limitations.

1. Dependence on internet connectivity.
2. Requirement for digital literacy among users.
3. Dependence on third-party cloud services.
4. Possible storage subscription costs.
5. Initial setup and training requirements.
6. Potential cybersecurity threats if not properly maintained.

These limitations may affect system accessibility, operational cost, and long-term maintenance requirements.

### **3.10 Summary of the Chapter**

This chapter presented the methodology and proposed system for the Cloud-Based Examination Script Management System. The Agile software development methodology was selected due to its flexibility, iterative nature, and ability to support continuous testing and feedback.

The chapter discussed the phases of the Agile process and the activities performed during each phase of development. It also presented the proposed system architecture, UML models, system features, development tools, and development environment.

The strengths and limitations of the proposed system were also analyzed. Overall, the proposed system is designed to improve examination script management through secure cloud storage, PDF compression, automated notifications, and efficient retrieval of academic records.